

SIMATS ENGINEERING

CO4_AT1- Code Challenge

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0613

NAME: C. HIMAJA

REG.NO: 192373035

Report

Title

Longest Common Subsequence (LCS) Using Dynamic Programming in C

Objective

To find the length of the longest subsequence common to two given strings using the Dynamic Programming technique.

Problem Description

Given two strings, find the longest subsequence that occurs in both strings while maintaining the same relative order.

The characters of the subsequence do not need to be contiguous.

Example

s1 = ABCDGH

s2 = AEDFHR

A common subsequence is:

ADH

Therefore:

LCS Length = 3

Algorithm

1. Read the two strings s1 and s2.

2. Find their lengths m and n .
3. Create a DP table of size $(m+1) \times (n+1)$.
4. Initialize the first row and first column with 0.
5. Compare characters from both strings.
6. If the characters are equal:

$$dp[i][j] = dp[i-1][j-1] + 1$$

7. If the characters are different:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

8. The value at $dp[m][n]$ is the length of the LCS.
9. Display the result.

DP Recurrence

When characters are equal

$$\text{if } s1[i-1] == s2[j-1]$$

$$dp[i][j] = dp[i-1][j-1] + 1$$

When characters are different

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

DP Table for the Given Example

For:

$s1 = \text{ABCDGH}$

$s2 = \text{AEDFHR}$

The final DP value is:

$$dp[6][6] = 3$$

Thus, the LCS length is 3.

Complexity Analysis

Complexity	Value
-------------------	--------------

Time Complexity	$O(m \times n)$
-----------------	-----------------

Complexity Value

Space Complexity $O(m \times n)$

Where:

- m = length of first string
- n = length of second string

Dynamic Programming avoids repeatedly solving the same subproblems, making it much more efficient than the basic recursive approach.

Output Analysis

Input

Enter first string: ABCDGH

Enter second string: AEDFHR

Output

LCS Length = 3

The common subsequence ADH occurs in both strings in the same relative order.

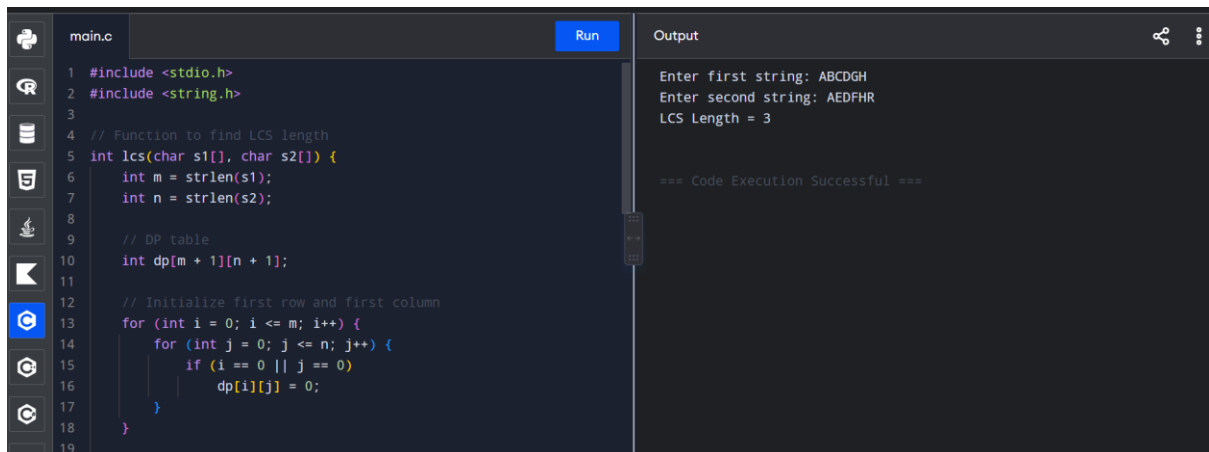
Therefore:

Maximum LCS Length = 3

Result

The Longest Common Subsequence problem was successfully implemented in C using Dynamic Programming. For the given strings ABCDGH and AEDFHR, the LCS length obtained is 3.

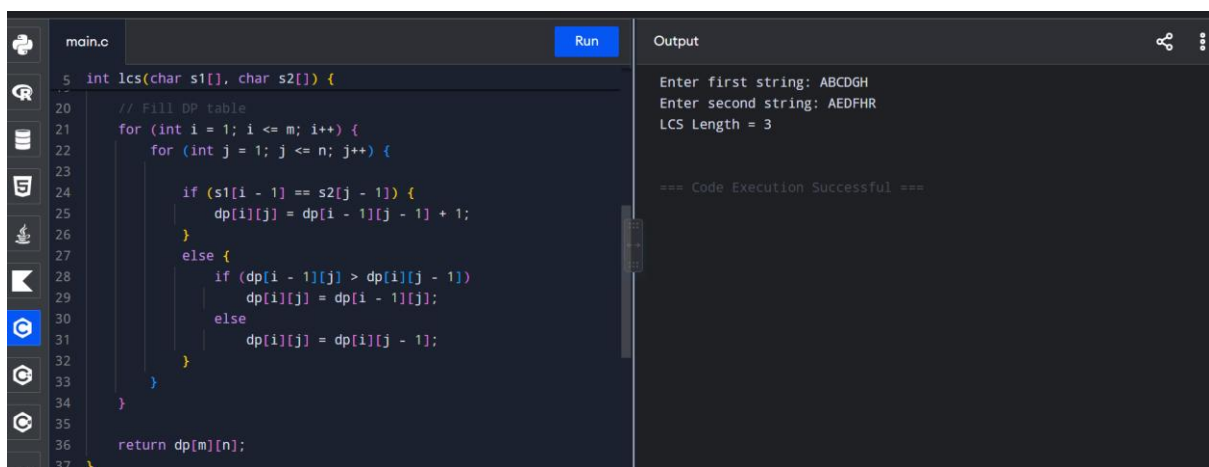
3. Screenshots for Report



```
main.c Run Output
1 #include <stdio.h>
2 #include <string.h>
3
4 // Function to find LCS length
5 int lcs(char s1[], char s2[]) {
6     int m = strlen(s1);
7     int n = strlen(s2);
8
9     // DP table
10    int dp[m + 1][n + 1];
11
12    // Initialize first row and first column
13    for (int i = 0; i <= m; i++) {
14        for (int j = 0; j <= n; j++) {
15            if (i == 0 || j == 0)
16                dp[i][j] = 0;
17        }
18    }
19}
```

Enter first string: ABCDGH
Enter second string: AEDFHR
LCS Length = 3

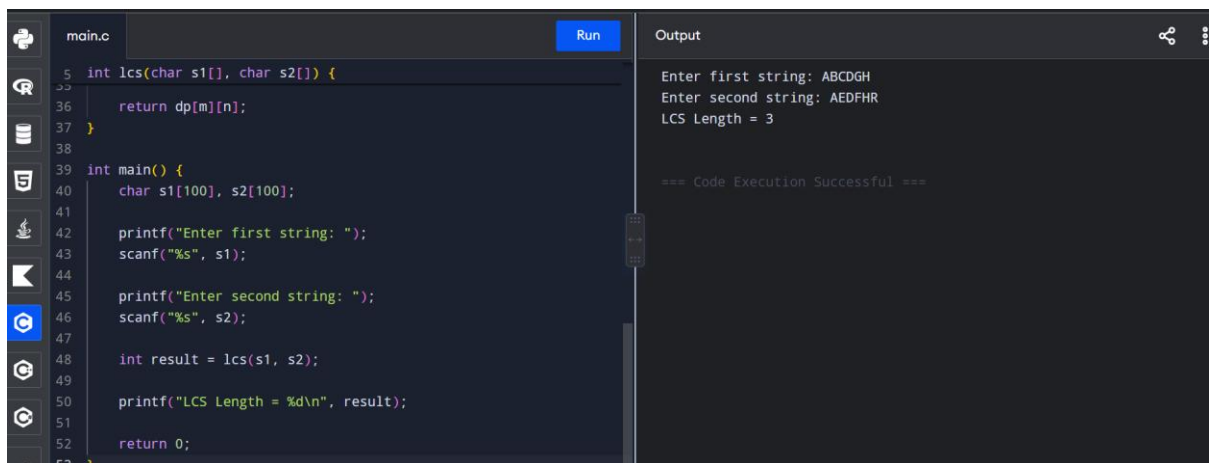
=== Code Execution Successful ===



```
main.c Run Output
5 int lcs(char s1[], char s2[]) {
20    // Fill DP table
21    for (int i = 1; i <= m; i++) {
22        for (int j = 1; j <= n; j++) {
23
24            if (s1[i - 1] == s2[j - 1]) {
25                dp[i][j] = dp[i - 1][j - 1] + 1;
26            }
27            else {
28                if (dp[i - 1][j] > dp[i][j - 1])
29                    dp[i][j] = dp[i - 1][j];
30                else
31                    dp[i][j] = dp[i][j - 1];
32            }
33        }
34    }
35
36    return dp[m][n];
37 }
```

Enter first string: ABCDGH
Enter second string: AEDFHR
LCS Length = 3

=== Code Execution Successful ===



```
main.c Run Output
5 int lcs(char s1[], char s2[]) {
36    return dp[m][n];
37 }
38
39 int main() {
40     char s1[100], s2[100];
41
42     printf("Enter first string: ");
43     scanf("%s", s1);
44
45     printf("Enter second string: ");
46     scanf("%s", s2);
47
48     int result = lcs(s1, s2);
49
50     printf("LCS Length = %d\n", result);
51
52     return 0;
53 }
```

Enter first string: ABCDGH
Enter second string: AEDFHR
LCS Length = 3

=== Code Execution Successful ===